

ForgeHash-X: A Custom-Sponge Memory-Hard Password Hashing Construction

Version 0 (Experimental Sandbox)

Thomas van Otterloo

Independent research / ForgeHash project
<https://github.com/ThomasBeHappy/Forgehash>

20 July 2026

Important. ForgeHash-X is **experimental research software**. It has not received independent cryptographic review. It must **not** be used to store production passwords. Prefer Argon2id, scrypt, bcrypt, or platform password APIs. This document is **not** a security proof. Digests are **not** compatible with ForgeHash-B3 (`$forgeh$v=1$...`).

Abstract

In the present work we introduce and systematically document **ForgeHash-X**, an experimental, clean-sheet memory-hard password hashing construction whose compression and expansion roles are discharged entirely by a custom sponge primitive (**ForgeX**), rather than by an externally standardized hash function such as BLAKE3. The construction is developed within the broader ForgeHash research program as a deliberately isolated sandbox line, identified in software and in encoded strings by the algorithm token `forgehx` and the version tag `v=0`. Heretofore, the project’s frozen conformance track—ForgeHash-B3—has explored memory-hard password hashing atop a modern extendable-output function; ForgeHash-X asks the complementary scientific question of what a fully in-house primitive stack entails when every BLAKE3-mediated role is replaced by a self-contained absorb–permute–squeeze interface.

Concretely, the design comprises three interlocking layers. First, ForgeX is a 1024-bit sponge over sixteen little-endian 64-bit words with rate $R = 8$ (512 bits) and capacity $C = 8$, driven by an eight-round ARX permutation **ForgePerm** whose quarter-round structure is inspired by, but cryptographically distinct from, ChaCha/BLAKE-family scheduling idioms. Second, a memory-hard fill operates on 512-byte blocks with password-dependent addressing, FastRange lane selection, and cross-lane synchronization every sixteen blocks—choices that deliberately diverge from the ForgeHash-B3 geometry. Third, a hostile-to-ambiguity canonical encoding of the form

`$forgehx$v=0$m,t,p$salt$hash$` binds parameters, salt, and digest under strict parse-and-re-encode verification. We provide a .NET 9 reference implementation (`ForgeHash.X.Core`) with no BLAKE3 dependency, frozen Toy-profile test vectors ($m = 1024$, $t = 1$, $p = 1$), and an empirical apparatus integrated with the project’s Collision Lab.

On 2026-07-20 we conducted uniqueness campaigns totaling 5×10^5 samples: four campaigns of 10^5 hashes each at Toy cost (RandomPairs, BitFlips, DistinctSalts, DistinctPasswords) and one additional DistinctPasswords campaign of 10^5 hashes at $m = 4096$ KiB, all with $t = 1$ and $p = 1$. Across these sample sets we observed zero accidental final-hash collisions (and zero seed collisions where tracked). We caution, however, that Toy costs remain far below interactive or sensitive deployment regimes; that uniqueness hunts are not birthday-bound collision searches at cryptographic width; and that we make no claim regarding resistance to GPU or ASIC adversaries, side channels, or time–memory tradeoffs. An informal throughput observation of approximately 2000 H/s at Toy on the author’s machine is reported solely as apparatus context and must not be compared to ForgeHash-B3 Development profiles that allocate 8 MiB matrices. The paper is offered as an open research artifact for specification scrutiny—not as a candidate production password KDF.

Keywords: password hashing, memory-hard functions, sponge constructions, ARX permutations, experimental cryptography, domain separation, credential stuffing, uniqueness campaigns

Contents

1	Introduction	6
1.1	Motivation and industrial context	6
1.2	The ForgeHash research program	6
1.3	Contributions	7
1.4	Paper organization	7
2	Related Work	8
2.1	A brief history of password hashing functions	8
2.2	Sponges, Keccak, and extendable output	8
2.3	ARX permutations: ChaCha, BLAKE, and BLAKE3	9
2.4	Domain separation and encoding hygiene	9
2.5	Time–memory tradeoffs and memory-hardness analyses	9
2.6	Positioning relative to ForgeHash-B3	9
3	Preliminaries and Notation	10
3.1	Basic notation	10
3.2	Sponge vocabulary	10
3.3	Memory-hard fill vocabulary	10
3.4	Encoding alphabet	10
3.5	Typographic conventions in this paper	10
4	Design Philosophy and Research Questions	11
4.1	Philosophy of the sandbox	11
4.2	Deliberate divergence from ForgeHash-B3	11
4.3	Research questions	11
4.4	Goals and non-goals	11
4.4.1	Goals	11
4.4.2	Non-goals (v0)	12
5	Threat Model	13
5.1	Assets and adversary capabilities	13
5.2	Offline guessing and credential stuffing	13
5.3	Multi-target attacks	13
5.4	Denial of service via parameters	13
5.5	Side channels	14
5.6	Explicitly out of scope	14
6	The ForgeX Sponge and ForgePerm	15
6.1	Design rationale for a custom sponge	15
6.2	State geometry	15
6.3	Domain separation	15
6.4	ForgePerm	15
6.4.1	Round constants	15
6.4.2	Quarter-round	16
6.4.3	Round body	16
6.5	Absorb, pad, squeeze	16
7	Memory-Hard Construction	17
7.1	Parameters and sandbox bounds	17
7.2	Seed derivation	17
7.3	Memory layout and initialization	17
7.4	Reference selection	17

7.5	Block mix	18
7.6	Fill schedule	18
7.7	Finalization	18
8	Encoding, Parsing Hostility, and Verification	19
8.1	Canonical string form	19
8.2	Parsing hostility as a design requirement	19
8.3	Verification	19
8.4	Non-compatibility with ForgeHash-B3	19
9	Implementation and Experimental Apparatus	20
9.1	Reference library	20
9.2	Tooling	20
9.3	Experimental apparatus for campaigns	20
9.4	Informal throughput context	20
10	Evaluation Methodology	21
10.1	What uniqueness hunts measure	21
10.2	Threats to validity	21
10.3	Unit-test methodology	21
10.4	Vector methodology	21
11	Results	22
11.1	Conformance vectors	22
11.2	Functional property tests	22
11.3	Empirical uniqueness campaigns	22
11.3.1	Toy cost (1024 KiB, $t = 1$, $p = 1$)	22
11.3.2	Higher-memory smoke (4096 KiB, $t = 1$, $p = 1$)	22
11.4	Throughput (informal)	22
12	Discussion	24
12.1	Interpreting a clean uniqueness log	24
12.2	Comparison with ForgeHash-B3	24
12.3	Position relative to OWASP guidance	24
12.4	On scientific style and “padding”	25
13	Limitations and Ethics	26
13.1	Technical limitations	26
13.2	Ethical considerations	26
14	Future Work	27
15	Conclusion	28
16	Acknowledgments	29
17	Extended Comparative Discussion	30
17.1	On the rhetoric of empirical uniqueness	30
17.2	On throughput narratives	30
18	Algorithmic Cost Sketch	31
19	Worked Example Walkthrough	32
20	Reproducibility and Artifact Checklist	33
21	Speculative Failure Modes (Non-Exhaustive)	34
22	Pedagogical Notes for Students	35
23	On Versioning Philosophy	36
24	Appendix A: Encoded Examples	37
25	Appendix B: Seed Digests (Toy Vectors)	38
26	Appendix C: Reproduce Campaigns	39

27	Appendix D: Ethical and Responsible Use Statement	40
28	Appendix E: Glossary and Notation Table	41
	References	42

1 Introduction

1.1 Motivation and industrial context

It is scarcely controversial to observe that password-based authentication continues to underwrite a substantial fraction of Internet identity systems, despite decades of advocacy for public-key credentials, hardware authenticators, and federated identity. When a credential database is disclosed—whether through application compromise, backup leakage, or insider misconduct—the residual security of user accounts hinges almost entirely on the cost an offline adversary must pay to invert stored password verifiers. Credential stuffing and large-scale password guessing campaigns make this cost concrete: adversaries combine leaked hash corpora with vast dictionaries, previously breached plaintext passwords, and highly parallel hardware. In such settings, a password hashing function (PHF) that is merely “cryptographically one-way” in the classical sense is often insufficient; the construction must also impose material sequential work and, preferably, substantial memory bandwidth and capacity, so that massively parallel guessing does not scale linearly with silicon budget alone.

Contemporary guidance therefore favors **memory-hard** password-based key derivation and hashing constructions. Percival’s script [1] demonstrated that sequential memory hardness could be engineered into practical password KDFs. Argon2 [2], emerging from the Password Hashing Competition [3], refined the engineering vocabulary of lanes, slices, data-dependent and data-independent addressing, and parameterized cost triples (m, t, p) . Balloon hashing [4] and related frameworks such as Catena [5] further enriched the design space and the associated theoretical analyses of time–memory tradeoffs. In parallel, operational documents such as the OWASP Password Storage Cheat Sheet [6] codify conservative deployment advice: prefer reviewed constructions (notably Argon2id), choose costs appropriate to the threat model, and avoid inventing password KDFs for production use. Against this backdrop, research into **new** PHF constructions occupies an awkward but legitimate niche. On the one hand, inventing a password hash and deploying it without review is widely recognized as hazardous. On the other hand, the scientific community benefits from carefully specified experimental artifacts: constructions that make design tradeoffs explicit, that ship bit-stable vectors, that instrument empirical uniqueness and avalanche behavior early, and that separate “research sandbox” versioning from any production recommendation. It is in this latter spirit that the ForgeHash project proceeds.

1.2 The ForgeHash research program

ForgeHash is an experimental project exploring memory-hard constructions with strong conformance tooling—frozen vectors, multi-language ports, mass uniqueness laboratories, and hostile parsers for encoded strings. Its first frozen line, **ForgeHash-B3**, uses BLAKE3 [7] for seed derivation, expansion, and finalization, together with a custom block mixer (**ForgeMix**). B3 answers a focused engineering question: **can we build a carefully specified, port-friendly PHF construction on a modern XOF, with digests that are stable across language implementations?** The encoded form `$forgeh$v=1$...` and the associated vector suites constitute a conformance baseline for that line.

ForgeHash-X asks a different and deliberately orthogonal question: **what does a fully custom stack look like if every BLAKE3 role is replaced by an in-house sponge?** The goals of version `v=0` are scientific and engineering clarity—not production readiness. We observe that replacing a mature external primitive with a young custom one increases cryptanalytic risk even if the surrounding memory-hard scaffolding is conventional. That risk is accepted here precisely

because the line is labeled experimental, encoded under a distinct algorithm identifier (`forgeh`), and documented with explicit non-goals.

1.3 Contributions

In the present paper we make the following contributions:

1. **Normative specification of ForgeX and ForgePerm.** We specify a 1024-bit sponge with rate eight words, capacity eight words, length-prefixed ASCII domain tags, sponge padding, and an eight-round ARX permutation ForgePerm, including round constants, quarter-round equations, column/diagonal scheduling, and a final word permutation.
2. **Memory-hard construction with deliberate geometric divergence from B3.** We specify a fill over 512-byte blocks (rather than B3’s 1024-byte blocks), with cross-lane cadence every sixteen blocks (rather than every thirty-two), FastRange-based foreign-lane selection, Argon2-like slice synchronization constraints, a ForgePerm-based block mix, and a multi-index XOR-fold finalization into a domain-separated output XOF.
3. **Canonical encoding and verification discipline.** We define the encoding `$forgeh$v=0$m,t,p$salt$hash$` with RFC 4648 Base64 (no padding), fixed field order, rejection of leading zeros and non-canonical forms, and constant-time digest comparison on verify. The X parser rejects B3 strings.
4. **Reference implementation and tooling.** We describe the .NET 9 library `ForgeHash.X.Core` (namespace `ForgeHashX`), which implements the full stack without any BLAKE3 package reference, together with Toy vectors under `implementers/x0/`, unit tests, and Collision Lab integration.
5. **Empirical uniqueness campaigns (2026-07-20).** We report four Toy campaigns of 10^5 samples each (RandomPairs, BitFlips, DistinctSalts, DistinctPasswords) and one DistinctPasswords campaign of 10^5 samples at $m = 4096$ KiB, for a total of 5×10^5 samples with zero observed accidental collisions in those sets. We accompany these results with an explicit threats-to-validity discussion.
6. **Research positioning.** We situate ForgeHash-X relative to PHC-era designs, sponge and ARX literature, TMTO analyses, OWASP guidance, and the ForgeHash-B3 conformance track, clarifying what may and may not be inferred from the present empirical baseline.

1.4 Paper organization

The remainder of this document is organized as follows. Section 2 surveys related work on password hashing functions, sponges, ARX permutations, domain separation, and time–memory tradeoffs. Section 3 establishes notation and preliminary definitions. Section 4 articulates design philosophy and research questions. Section 5 expands the informal threat model. Section 6 and Section 7 specify ForgeX/ForgePerm and the memory-hard construction, respectively. Section 8 treats encoding, parsing hostility, and verification. Section 9 describes the reference implementation and experimental apparatus. Section 10 and Section 11 present evaluation methodology and results. Section 12 interprets findings and positions the work relative to B3 and OWASP guidance. Section 13, Section 14, and Section 15 discuss limitations and ethics, future work, and conclusions. Acknowledgments follow, after which Appendices A–E collect encoded examples, seed digests, reproduction instructions, an ethical statement, and a glossary of notation.

2 Related Work

2.1 A brief history of password hashing functions

Password hashing has evolved through several generations of engineering practice. Early systems often stored unsalted or lightly salted digests of passwords under general-purpose hash functions, a posture that proved catastrophically inadequate once GPU and botnet cracking became inexpensive. PBKDF2 [8] introduced iterated keyed hashing as a standardized mechanism for stretching passwords, and it remains widely deployed; nevertheless, its memory footprint is modest, and highly parallel adversaries can often amortize computation effectively. bcrypt [9] improved practical resistance by incorporating a cost parameter and a design historically tied to the Blowfish key schedule, imposing sequential work that was, for its era, comparatively expensive to accelerate. Yet bcrypt’s memory usage is still limited relative to later memory-hard proposals, and modern guidance typically prefers Argon2id for new systems while acknowledging bcrypt’s long operational track record [6].

Percival’s scrypt [1] marked a decisive conceptual shift: by forcing large-memory sequential mixing (ROMix), scrypt raised the capital cost of large-scale guessing on memory-constrained parallel hardware. The Password Hashing Competition [3] subsequently catalyzed a wave of designs and analyses, culminating in Argon2 [2] as the winner and de facto recommendation for many deployments. Argon2’s parameterization—memory m , iterations t , parallelism p —together with its lane/slice fill structure and its data-dependent (Argon2d), data-independent (Argon2i), and hybrid (Argon2id) modes, established a shared vocabulary that subsequent experimental constructions, including ForgeHash-X, inherit at the engineering level even when they replace the underlying compression primitive.

Balloon hashing [4] contributed a memory-hard framework with an emphasis on provable aspects of sequential attack resistance under idealized assumptions, while Catena [5] explored password scrambling with explicit attention to memory consumption and cache-timing considerations. We do not claim that ForgeHash-X matches the analytical maturity of these lines; rather, we note that they define the intellectual neighborhood in which a new experimental PHF must situate itself. It is worth noting that “memory-hard” is not a single binary property but a family of heuristics and formalizations concerning how an adversary’s time and memory may be traded; informal RAM-filling arguments are necessary but not sufficient for cryptographic confidence.

2.2 Sponges, Keccak, and extendable output

The sponge construction [10] provides a unified absorb–permute–squeeze paradigm that underlies Keccak and the SHA-3 family [11], as well as numerous extendable-output functions (XOFs). In a sponge, a fixed-width state is partitioned into a rate region, which interfaces with input and output, and a capacity region, which is intended to retain secret mixing mass against adversarial observation of rate bytes. Domain separation—whether by padding rules, capacity conventions, or explicit tags—plays a central role in preventing trivial cross-protocol collisions when the same permutation is reused in multiple modes [12].

ForgeX adopts the sponge interface as its sole primitive surface for seed derivation, block expansion, finalization hashing, and output stretching. We emphasize that this choice is architectural rather than a claim of SHA-3-equivalent security: ForgePerm is a young ARX permutation without third-party cryptanalysis, and the sponge parameters (1024-bit state, 512-bit rate) are selected for implementation convenience and research clarity within the ForgeHash-X sandbox, not as an assertion of optimal capacity margins for password hashing.

2.3 ARX permutations: ChaCha, BLAKE, and BLAKE3

Add–rotate–XOR (ARX) designs have been a dominant motif in software-oriented symmetric cryptography. ChaCha [13] refined the Salsa20 lineage with a quarter-round structure that maps efficiently onto general-purpose CPUs and admits comparatively straightforward constant-time implementations. BLAKE [14] and its successors carried ARX ideas into the hash-function setting; BLAKE3 [7] further optimized tree hashing and XOF usage for modern hardware, becoming the primitive substrate of ForgeHash-B3.

ForgePerm is **inspired by** the column/diagonal scheduling idiom familiar from ChaCha/BLAKE-family designs, but it employs distinct round constants, a distinct quarter-round—including 32×32 products folded into the ARX update—and a distinct word permutation after each round’s quarter-rounds. No security reduction to ChaCha, BLAKE, or BLAKE3 is claimed or intended. In the present work, the ARX choice is motivated by implementability in managed runtimes (.NET), readability of the specification, and avoidance of table-driven S-boxes that complicate constant-time discussion for the permutation core itself. We reiterate that the surrounding memory fill is intentionally data-dependent and therefore outside any simplistic “constant-time PHF” claim.

2.4 Domain separation and encoding hygiene

Rogaway’s perspective on nonce-based encryption and, more broadly, the culture of explicit domain separation [12] inform our treatment of tags and encoded strings. Password hashes that circulate as strings are particularly exposed to parser differentials: ambiguous Base64, whitespace tolerance, leading zeros in cost parameters, and accidental acceptance of foreign algorithm identifiers can all produce security or interoperability failures. ForgeHash-X therefore treats the encoded form as a first-class protocol object: the algorithm id `forgehx`, the version `v=0`, the parameter triple, and the salt/digest payloads are bound under a re-encode-to-canonical check. This stance is engineering discipline rather than a novel cryptanalytic result, but it is essential for multi-language ports and for preventing silent confusion with `$forgeh$` digests.

2.5 Time–memory tradeoffs and memory-hardness analyses

Alwen and Blocki [15] and related lines of work have clarified that memory-hard functions must be evaluated not only by peak memory in the honest algorithm, but also by how an adversary might reduce memory at the expense of increased computation or depth. Data-independent fills admit certain parallel evaluation strategies; data-dependent fills can frustrate some tradeoffs while introducing cache-timing and secret-dependent memory-access concerns. ForgeHash-X v0 employs password-dependent addressing in the style of many practical PHFs, with Argon2-like synchronization constraints for cross-lane reads. We make **no** TMTO lower-bound claim. The citation of Alwen–Blocki here is contextual: it frames why “we filled m KiB” is not, by itself, a theorem.

2.6 Positioning relative to ForgeHash-B3

ForgeHash-B3 (`$forgeh$v=1$`) remains the project’s frozen BLAKE3-based construction with official vectors and multi-language ports. ForgeHash-X is intentionally **non-compatible**: different algorithm identifier, version `v=0`, different block size (512 versus 1024 bytes), different cross-lane cadence (every 16 versus every 32 blocks), different mix and finalization, and zero shared digest namespace. B3 answers the portability-and-conformance question atop a reviewed XOF; X answers the clean-sheet-primitive question inside a sandbox. It is worth noting that neither line is recommended for production password storage in the absence of extensive independent review; B3’s greater implementation maturity does not equate to a production endorsement.

3 Preliminaries and Notation

3.1 Basic notation

Unless otherwise stated, integers are non-negative. We write $a \oplus b$ for bitwise exclusive-or, $a + b$ for wrapping addition in the ambient word type, and $\text{rotl64}(x, n)$ / $\text{rotr64}(x, n)$ for 64-bit rotations. The product notation $a \times b$ denotes wrapping multiplication in `u64` unless a wider product is explicitly indicated. For a byte string x , $|x|$ denotes its length in bytes. $\text{LE32}(n)$ and $\text{LE64}(n)$ denote 32- and 64-bit little-endian encodings of integers. Concatenation is written $x \parallel y$.

A **password** is an arbitrary byte string accepted by the API (subject to implementation limits). A **salt** is a byte string of length between 16 and 64 bytes inclusive in the `v0` sandbox. Parameters are a triple (m, t, p) where m is memory in KiB, t is the iteration (pass) count, and p is parallelism (lane count), together with an `outputLength` in bytes.

3.2 Sponge vocabulary

A sponge maintains a state S of b bits, partitioned into rate R and capacity C with $b = R + C$. Absorption XORs input into the rate and applies a permutation f when the rate is full. Squeezing emits rate bytes, applying f as needed. ForgeX instantiates $b = 1024$, $R = 512$, $C = 512$, and $f = \text{ForgePerm}$. We write `ForgeX-Hash(tag, data)` for absorb-domain-and-data, pad, squeeze 32 bytes; and `ForgeX-XOF(tag, data, n)` for the analogous n -byte squeeze.

3.3 Memory-hard fill vocabulary

The construction allocates a matrix of **blocks** of 512 bytes each. Blocks are organized into p **lanes**. The fill proceeds over t **passes**, each divided into four **slices**. Reference selection may read a prior block from the same lane or, under stated cadence and synchronization constraints, from a foreign lane. The honest algorithm’s peak memory is proportional to m KiB; we do not formalize adversarial memory complexity in this document.

3.4 Encoding alphabet

Encoded strings use ASCII `$` as a field separator and RFC 4648 Base64 without padding for binary fields. The algorithm token is the literal `forgeh`. Version strings in `v0` are exactly `v=0`. Parameter fields appear as `m=<int>,t=<int>,p=<int>` without spaces or leading zeros.

3.5 Typographic conventions in this paper

Algorithm identifiers that contain dollar signs are never written as raw math markup; they appear via `#raw(...)` so that Typst does not interpret $\$. \dots \$$ as equation delimiters. Experimental caveats are highlighted with the `warn` box defined in the document preamble. All empirical claims in Section 11 refer to campaigns dated 2026-07-20 unless explicitly restated.

4 Design Philosophy and Research Questions

4.1 Philosophy of the sandbox

ForgeHash-X is developed under an explicit **sandbox philosophy**. The version tag `v=0` signals that digests may break without a migration story; the algorithm id `forgeh` prevents accidental interoperability with B3; and the documentation repeatedly states that the construction is not a production KDF. This posture is not theatrical humility—it is a design constraint. By refusing compatibility and production framing, we grant ourselves permission to change block geometry, mix structure, and even the permutation if research findings demand it, while still shipping bit-stable Toy vectors for the current snapshot.

We further adopt a **specification-first** posture: endianness, domain tags, padding bytes, addressing rules, and encoding canonicalization are treated as normative. Empirical labs and unit tests are instrumentation around that specification, not substitutes for it. In our view, many amateur PHF proposals fail less from lack of “clever rounds” than from underspecified corner cases that cause ports to diverge silently.

4.2 Deliberate divergence from ForgeHash-B3

A subtle failure mode in evolving a research PHF family is accidental near-duplication: changing a constant or two while retaining the same primitive and geometry, then over-claiming novelty. ForgeHash-X therefore diverges along multiple axes simultaneously: no BLAKE3 dependency; 512-byte rather than 1024-byte blocks; cross-lane synchronization every 16 rather than every 32 blocks; a ForgePerm-based mix rather than ForgeMix; and a distinct finalization graph. The intent is that an analyst comparing B3 and X should not conclude that X is “B3 with BLAKE3 swapped out.” It is a different construction in the same research program.

4.3 Research questions

We organize the scientific agenda of v0 around the following questions:

1. **RQ1 (Specificity)**. Can a fully custom sponge-plus-fill PHF be specified with sufficient precision that a .NET reference and frozen vectors determine a unique Toy digest stream for ports?
2. **RQ2 (Separation)**. Do distinct passwords, salts, and parameter tuples yield distinct digests in large but sub-birthday random sample sets at Toy and modestly higher memory, as a basic functional smoke test?
3. **RQ3 (Engineering coupling)**. Can Collision Lab and shared campaign machinery developed for B3 be reused for X without digest-namespace confusion?
4. **RQ4 (Non-claim discipline)**. Can the project report empirical results without sliding into unjustified security conclusions about ForgePerm or memory-hardness?

The present paper answers RQ1 affirmatively at the level of the current reference and Toy vectors; offers encouraging but narrowly scoped evidence on RQ2; demonstrates RQ3 via lab integration; and attempts to practice RQ4 in the discussion and limitations sections. We do not claim to answer the deeper cryptanalytic questions that would be required for any production discussion.

4.4 Goals and non-goals

4.4.1 Goals

1. **Self-contained primitive**. No BLAKE3 (or other external hash) dependency in the X core.
2. **Normative clarity**. Endianness, domain separation, padding, addressing, and encoding are fully specified for bit-exact reproduction.

3. **Deliberate divergence from B3.** Avoid accidental “BLAKE3 with different parameters” by changing block geometry, mix, and cross-lane cadence.
4. **Research tooling.** Toy vectors, unit tests, and Collision Lab integration for early empirical feedback.

4.4.2 Non-goals (v0)

1. Production password storage or cryptographic certification.
2. Compatibility with `$forgeh$` digests.
3. GPU/ASIC evaluation, cache-timing claims, or side-channel hardening proofs.
4. A formal indistinguishability or memory-hardness theorem for ForgePerm / ForgeHash-X.

Important. Meeting the goals above—and observing zero collisions in large Toy campaigns—does **not** imply that ForgeHash-X is safe. It shows specification stability and basic functional separation under those sample sets.

5 Threat Model

5.1 Assets and adversary capabilities

We consider, informally, the standard offline password-guessing adversary. The adversary obtains one or more encoded hash strings, each containing parameters (m, t, p) , a salt, and a digest (and, implicitly, the algorithm and version identifiers). The adversary may allocate substantial computation and memory, may attempt to optimize the honest fill with alternative schedules, and may exploit parallelism across guesses. The defender’s asset is the confidentiality of the password (and, secondarily, integrity of the verification decision).

Desired properties—aspirational for a future reviewed version, and **not** theorems of the present work—include:

- **One-wayness under guessing:** given salt and parameters, producing a password that verifies should require work comparable to honest evaluation per guess, barring cryptanalytic shortcuts in ForgePerm or the fill.
- **Salted uniqueness:** distinct salts should yield unrelated digests for the same password, preventing trivial rainbow precomputation across users.
- **Parameter binding:** digests should depend on $(m, t, p, \text{outputLength})$ so that parameter downgrade or mismatch is detectable upon honest verification.
- **Memory-hardness heuristics:** filling a large matrix with data-dependent references should raise sequential RAM cost for the honest-class adversary.

5.2 Offline guessing and credential stuffing

Offline guessing is the central in-scope threat. Credential stuffing—replaying password lists across services—is primarily mitigated by unique salts and by service-side rate limiting for **online** attempts; the PHF’s role is to ensure that a stolen hash file does not immediately yield plaintext passwords at trivial cost. We observe that PHF cost parameters interact poorly with naive service-side latency budgets: operators sometimes choose Toy-like costs for “snappy login,” thereby collapsing offline hardness. ForgeHash-X’s Toy profile exists for vectors and labs; shipping Toy in an authentication path would be reckless irrespective of uniqueness-hunt outcomes.

5.3 Multi-target attacks

At Internet scale, an adversary may obtain N salted hashes and attempt to amortize work across targets (e.g., testing a popular password against many salts). Salting frustrates precomputation, but multi-target settings still motivate adequate per-guess cost and adequate digest width. Our uniqueness campaigns at $N = 10^5$ per campaign are **not** multi-target attack simulations; they are functional separation smokes. We explicitly place Internet-scale multi-target security arguments out of scope for v0 claims.

5.4 Denial of service via parameters

Encoded parameters create a DoS surface: a maliciously large m or t could cause verifiers to allocate excessive memory or CPU. The v0 sandbox defines bounds ($m \in [256, 65536]$ KiB, $t \in [1, 8]$, $p \in [1, 16]$, etc.) and the reference rejects absurd inputs. We do not claim a complete resource-control architecture for untrusted parameter sources (e.g., hash strings supplied by attackers in online protocols); deployments that accept encoded hashes from untrusted parties require additional policy beyond this paper.

5.5 Side channels

The ForgePerm ARX core avoids data-dependent branches in the permutation itself. The memory fill, however, is intentionally data-dependent in reference selection, and thus may exhibit password-dependent memory-access patterns. Cache timing, memory-bus contention, and related side channels are out of scope for v0 security claims. We mention them so that readers do not infer side-channel robustness from ARX rhetoric alone.

5.6 Explicitly out of scope

The following are out of scope for v0 claims:

- Quantum adversaries and Grover-style cost adjustments.
- White-box mobile attackers with perfect timing and instrumentation.
- Formal TMTO lower bounds in the Alwen–Blocki sense [15].
- GPU/ASIC cost models or “joules per guess” measurements.
- Social engineering, phishing, and online guessing without database theft.
- Compatibility with or migration from ForgeHash-B3 digests.

6 The ForgeX Sponge and ForgePerm

6.1 Design rationale for a custom sponge

We adopt a sponge as the sole symmetric primitive surface for several reasons. First, sponges naturally support both fixed-length hashing and XOF-style expansion, which the construction needs for seed derivation (32 bytes), initial block expansion (512 bytes), final root hashing (32 bytes), and output stretching (`outputLength` bytes). Second, a single permutation keeps the cryptanalytic target concentrated: analysts need not reason about a menagerie of unrelated compression functions. Third, implementing a sponge in .NET without native dependencies simplifies the research sandbox’s reproducibility story.

These rationales do **not** justify skipping review. A custom sponge concentrates risk: a break in ForgePerm potentially undermines seed, expand, final, and output stages simultaneously. Domain separation tags mitigate **trivial** cross-mode confusion but do not create security reductions.

6.2 State geometry

ForgeX maintains a state of 16 little-endian 64-bit words (1024 bits). The rate is $R = 8$ words (512 bits / 64 bytes); the capacity is $C = 8$ words. Rate bytes occupy $S[0..7]$. The initial state is all zeros. Each convenience call (`ForgeX-Hash`, `ForgeX-XOF`) starts from a fresh zero state.

It is worth noting that a 512-bit capacity is large relative to the 256-bit outputs used in Toy vectors; we do not claim that this margin was derived from a tight password-hashing cost model. The geometry was chosen for alignment with 64-byte rate blocks and 16-word ForgePerm states, favoring implementability and clarity in v0.

6.3 Domain separation

ASCII domain tags are absorbed as length-prefixed bytes: `LE32(len(tag)) || tag`. This length prefix avoids ambiguity if tag alphabets were ever extended to non-self-describing forms. Tags used by the construction are listed in Table 1.

Tag	Role
<code>ForgeX/v0/seed</code>	Password/parameter seed (32-byte squeeze)
<code>ForgeX/v0/expand</code>	Initial memory block XOF (512 bytes)
<code>ForgeX/v0/final</code>	Final root (32-byte squeeze)
<code>ForgeX/v0/output</code>	Output XOF (<code>outputLength</code> bytes)

Table 1: Domain tags for ForgeHash-X v0.

The version fragment `/v0/` inside tags is intentional: should a future `v=1` of ForgeHash-X redesign the sponge usage, tags can change in lockstep with encoded versioning, reducing the risk of accidental cross-version domain reuse.

6.4 ForgePerm

ForgePerm is an 8-round in-place permutation on the 16-word state. Eight rounds is a research-era choice reflecting a desire for a compact specification and fast Toy-cost experimentation; it is **not** the output of a published third-party cryptanalysis recommending a round count. Heretofore we treat ForgePerm as an opaque idealized permutation only in informal discussion; in claims we treat it as a concrete, unreviewed ARX design.

6.4.1 Round constants

For round $r \in \{0, \dots, 7\}$ and word index $i \in \{0, \dots, 15\}$:

$$\text{RC}[r][i] = \text{rotl64}(c_0 \oplus (r \times c_1) \oplus (i \times c_2), (r + 3i) \bmod 64)$$

with constants $c_0 = 0x9E3779B97F4A7C15$, $c_1 = 0xD1B54A32D192ED03$, $c_2 = 0xA24BAED4963EE407$.

Multiplication is wrapping `u64`. At the start of round r , each word is XORed with the corresponding round constant. The constants are “nothing-up-my-sleeve” style words derived from familiar irrational-related patterns; we do not claim that this derivation yields cryptographic unpredictability beyond standard engineering practice for ARX round constants.

6.4.2 Quarter-round

On word indices (a, b, c, d) , letting `Low32` denote the low 32 bits of a word, and using wrapping arithmetic:

```

a ← a + b + 2*Low32(a)*Low32(b)
d ← rotr64(d ⊕ a, 17)
c ← c + d + 2*Low32(c)*Low32(d)
b ← rotr64(b ⊕ c, 11)
a ← a + b + 2*Low32(a)*Low32(b)
d ← rotr64(d ⊕ a, 23)
c ← c + d + 2*Low32(c)*Low32(d)
b ← rotr64(b ⊕ c, 41)

```

The inclusion of $2 \times \text{Low32}(x) \times \text{Low32}(y)$ terms strengthens nonlinear mixing relative to pure ARX addition chains, at the cost of departing from any security proof tied to ChaCha’s quarter-round. Rotation distances 17, 11, 23, and 41 are fixed and part of the normative definition.

6.4.3 Round body

After constant injection:

1. Column quarter-rounds on $(0, 4, 8, 12)$, $(1, 5, 9, 13)$, $(2, 6, 10, 14)$, $(3, 7, 11, 15)$.
2. Diagonal quarter-rounds on $(0, 5, 10, 15)$, $(1, 6, 11, 12)$, $(2, 7, 8, 13)$, $(3, 4, 9, 14)$.
3. Word permutation: $T[(7i + 3) \bmod 16] \leftarrow S[i]$, then $S \leftarrow T$.

The column/diagonal pattern echoes ChaCha-family diffusion scheduling; the final word permutation is an X-specific diffusion step intended to reduce fixed-position inertia across rounds. We invite cryptanalysis of the combined structure; we do not assert optimal diffusion bounds.

6.5 Absorb, pad, squeeze

Absorbing XORs input bytes into the rate; a full rate triggers `ForgePerm` and resets the rate offset. Before the first squeeze:

```

rate[offset] ⊕= 0x01
rate[63]      ⊕= 0x80
ForgePerm(S); offset ← 0

```

Squeeze copies rate bytes, permuting when the rate is exhausted. This padding pattern follows a sponge-style multi-rate padding intuition (domain bit in the first free rate byte and a high-bit marker at the end of the rate block). Exact bytes are normative; ports must not substitute “equivalent” padding heuristics.

Convenience APIs:

- `ForgeX-Hash(tag, data) → absorb domain+data, pad, squeeze 32 bytes.`
- `ForgeX-XOF(tag, data, n) → absorb domain+data, pad, squeeze  $n$  bytes.`

7 Memory-Hard Construction

7.1 Parameters and sandbox bounds

Sandbox bounds (v0) are summarized in Table 2.

Parameter	Symbol	Sandbox range
Memory	m (memoryKiB)	256 ... 65536 KiB
Iterations	t	1 ... 8
Parallelism	p	1 ... 16
Output length	outputLength	16 ... 64 bytes
Salt length	—	16 ... 64 bytes

Table 2: Sandbox parameter bounds for ForgeHash-X v0.

Constraints: $m \times 1024$ divisible by block size 512; $\text{blockCount} = m \times 1024 / 512$ divisible by p ; blocks-per-lane divisible by 4 and at least 8.

These bounds are research-sandbox limits for the reference, not endorsements of any particular deployment cost. **Toy profile** (vectors and mass hunts in this paper): $m = 1024$, $t = 1$, $p = 1$, outputLength = 32, salt length 16.

7.2 Seed derivation

```
material =
  LE32(0) || LE32(m) || LE32(t) || LE32(p) || LE32(outputLength)
  || LE64(|password|) || password || LE32(|salt|) || salt
seed = ForgeX-Hash("ForgeX/v0/seed", material) // 32 bytes
```

Rationale. The leading $\text{LE32}(0)$ reserves a versioning/domain niche inside the seed material itself, complementary to the ASCII tag. Explicit lengths before password and salt bytes prevent canonicalization ambiguities between adjacent fields. Binding $(m, t, p, \text{outputLength})$ into the seed ensures that parameter changes alter the entire fill, not merely a final tweak. We observe that seed binding is necessary but not sufficient for parameter security: verifiers must still parse and enforce encoded parameters consistently.

7.3 Memory layout and initialization

Blocks are 512 bytes ($64 \times u64$). Storage is lane-major. For each lane L and $i \in \{0, 1\}$:

```
block[L][i] = ForgeX-XOF("ForgeX/v0/expand", seed || LE32(L) || LE32(i), 512)
```

Rationale for 512-byte blocks. Halving B3’s 1024-byte block size increases the number of blocks for a given m , altering the addressing graph’s granularity and the frequency of mix invocations. This is a deliberate geometric fork, not an optimization claim. Initializing two blocks per lane via XOF provides a password-dependent starting surface before the data-dependent fill begins at index 2 in pass 0 / slice 0.

7.4 Reference selection

Let prev be the previous block in the current lane. An address word mixes selected words with pass and block index. For $p > 1$, every 16th block may select a foreign lane via $\text{FastRange}(\text{prev}[1], p)$; otherwise the reference stays in-lane. $\text{FastRange}(x, n)$ is the high 64 bits of the 128-bit product $x \times n$ (Lemire-style mapping [16]). Allowed index regions respect pass-0 prefixes and completed slices for cross-lane reads (an Argon2-like synchronization story with X-specific constants).

Rationale. Data-dependent addressing is a standard practical PHF ingredient: it couples the password into the memory-access pattern, frustrating certain precomputation strategies that assume

a fixed access graph. Cross-lane cadence every 16 blocks increases inter-lane coupling frequency relative to B3's every 32 blocks, again as a deliberate divergence. FastRange provides a uniform-ish map from a 64-bit word to $\{0, \dots, n - 1\}$ without rejection sampling loops that would complicate constant-iteration discussion in the lane-index path. We do not claim that these choices yield optimal TMTO resistance.

7.5 Block mix

Each 64-word block is split into four 16-word chunks. Chunk k injects pass, lane, block index, and a rotated mix of those values into the first words, applies ForgePerm, and feed-forwards with XOR against the pre-permutation chunk.

Rationale. Injecting positional metadata before ForgePerm ties the mix to the fill coordinates, reducing the risk that identical block contents at different indices behave as interchangeable. Feed-forward XOR preserves dependency on the pre-mix chunk. Using ForgePerm here—rather than a separate mixer primitive—keeps the cryptanalytic surface unified, at the cost of concentrating risk on ForgePerm's differential and linear properties.

7.6 Fill schedule

For each pass, four slices, then each lane (sequential or parallel with a barrier after each slice): combine previous XOR reference, mix, and write (pass 0) or XOR-accumulate (later passes). Pass 0 / slice 0 starts at block index 2 because blocks 0–1 were initialized by XOF.

Rationale. The four-slice structure and lane barriers follow the broad Argon2-era engineering pattern that enables p -way parallelism without fully decoupling lanes. XOR-accumulation on later passes increases dependency density across t . Starting at index 2 is a simple consequence of XOF initialization; ports that erroneously overwrite blocks 0–1 during fill will fail vectors.

7.7 Finalization

XOR-fold selected blocks per lane (last, quarter, half, three-quarter indices), hash with the `final` tag to a 32-byte root, then XOF with the `output` tag to `outputLength` bytes:

```
output = ForgeX-XOF("ForgeX/v0/output", root || seed, outputLength)
```

Rationale. Multi-index folding reduces reliance on a single terminal block, which might otherwise become a narrow cut point. Re-binding `seed` into the output XOF ties the final bytes to the parameter/password/salt seed material even if an implementer mistakenly truncated intermediate state. The split between `final` (32-byte root) and `output` (variable-length XOF) mirrors the hash-then-XOF pattern common in sponge-based designs and keeps domain tags distinct.

8 Encoding, Parsing Hostility, and Verification

8.1 Canonical string form

Canonical string:

```
$forgehx$v=0$m=<memoryKiB>,t=<iterations>,p=<parallelism>$<salt-b64>$<hash-b64>
```

Base64 is RFC 4648 without padding. Field order is fixed; leading zeros in integers are rejected; the parser re-encodes and demands byte-identical canonical form. The X parser rejects `$forgeh$ / v=1` strings.

8.2 Parsing hostility as a design requirement

We use the term **parsing hostility** for a deliberate stance: inputs that are merely “close” to canonical form are rejected rather than normalized away. This includes whitespace infiltration, alternative Base64 alphabets, padded Base64, reordered parameter keys, and leading zeros such as `m=01024`. The motivation is twofold. First, security protocols that normalize aggressively invite language-specific divergences (e.g., one port accepts spaces, another does not), producing interoperability failures that masquerade as “flaky tests.” Second, ambiguous encodings complicate logging, auditing, and migration tooling.

It is worth noting that parsing hostility can frustrate operators who hand-edit hash strings; we consider that an acceptable cost inside a research construction that aims at bit-exact ports. Production password APIs often hide encodings behind opaque blobs or standardized PHC strings; our encoded form is an explicit, inspectable research artifact.

8.3 Verification

Verification recomputes the digest from password, salt, and parameters and compares against the encoded hash in constant time (byte-wise, length-checked). Error messages in the reference are designed not to distinguish “salt decode failed” from “digest mismatch” in APIs that are exposed to untrusted online clients; the present paper does not mandate a particular error taxonomy beyond recommending that online verifiers avoid password-oracle side channels in their application logic.

8.4 Non-compatibility with ForgeHash-B3

Because B3 and X share a superficial “dollar-separated PHF string” aesthetic, we emphasize algorithmic segregation: an X verifier must not fall through to B3 parsing, and vice versa. Distinct algorithm tokens (`forgehx` vs `forgeh`) and version disciplines exist precisely so that mixed databases fail closed.

9 Implementation and Experimental Apparatus

9.1 Reference library

The .NET 9 project `ForgeHash.X.Core` (namespace `ForgeHashX`) implements `ForgeX`, `ForgePerm`, the memory engine, encoding, and the public API:

- `DeriveHash` / `DeriveHashParallel` (lane-parallel fill, sequential must match),
- `ComputeSeed`, `HashPassword`, `VerifyPassword`.

There is **no** Blake3 package reference. This absence is load-bearing for RQ1: the X core must be understandable and portable without pulling in the B3 primitive stack. Managed code also makes endianness and wrapping arithmetic explicit, which we consider advantageous for a specification-driven sandbox even if peak throughput is lower than highly tuned native code.

9.2 Tooling

- Toy vectors: `implementers/x0/` (+ `tools/ForgeHash.X.VectorGen`).
- Tests: `tests/ForgeHash.X.Tests` (vectors, determinism, parameters, avalanche, parallel equivalence, encoding, collision smokes).
- Collision Lab: `src/ForgeHash.CollisionLab` algorithm selector **ForgeHash-X** with Toy preset, backed by shared `CollisionCampaign` and `XCollisionHasher` in `ForgeHash.Analysis`.

9.3 Experimental apparatus for campaigns

Uniqueness campaigns reported herein were executed via Collision Lab on 2026-07-20. Unless noted, campaigns used $N = 100,000$ samples, workers approximately equal to CPU count on the author’s machine, Toy preset ($m = 1024$, $t = 1$, $p = 1$) or an explicit $m = 4096$ override for the higher-memory smoke. Tracks recorded final hashes and, where stated, seeds. The apparatus is shared with the B3 research track at the campaign-orchestration layer; hasher backends are algorithm-specific to prevent digest-namespace confusion.

9.4 Informal throughput context

Mass runs at Toy reported on the order of approximately 2000 hashes/s on the author’s machine. We stress—twice, because misreading this figure is easy—that the number is **dominated by the 1 MiB Toy matrix fill**, not by a claim that ForgeX “beats BLAKE3.” ForgeHash-B3 Development uses 8 MiB matrices (about $8 \times$ more memory). Matched- (m, t, p) benchmarks remain future work and are required before any comparative performance narrative.

10 Evaluation Methodology

10.1 What uniqueness hunts measure

A uniqueness hunt samples inputs from a stated distribution, computes digests (and optionally seeds), and searches for collisions within the sample set. In our RandomPairs campaigns, both password and salt vary. In BitFlips, passwords are nearby in Hamming distance. In DistinctSalts, the password is fixed and salts vary. In DistinctPasswords, the salt is fixed and passwords vary. A result of zero collisions indicates that, **within that sample set**, the reference did not map two distinct sampled inputs to the same tracked digest.

This measurement is useful as a **functional smoke**: it catches gross bugs such as ignored salts, truncated states, or accidental constant outputs. It is also weak as a **cryptographic claim**: at 256-bit digest width, birthday bounds [17] imply that 10^5 samples are negligible as a collision search. Clean results are therefore best interpreted as “no smoking gun in the lab,” not as evidence that ForgePerm is collision-resistant.

10.2 Threats to validity

We enumerate principal threats to validity:

1. **Sub-birthday sample size.** 5×10^5 total samples cannot stress 256-bit collision resistance.
2. **Distribution narrowness.** Synthetic random and bit-flip distributions do not model adversarial structured passwords crafted to exploit ForgePerm differentials.
3. **Single-implementation bias.** Campaigns exercise the .NET reference; a buggy port could still collide or diverge.
4. **Cost narrowness.** Four of five campaigns use Toy ($m = 1024$); one uses $m = 4096$. Neither approaches interactive/sensitive Argon2id deployments.
5. **Environment specificity.** Throughput remarks are single-machine and informal.
6. **Confirmation bias risk.** Researchers may overweight zero-collision headlines; we attempt to counteract this in Section 12 and Section 13.

10.3 Unit-test methodology

Beyond campaigns, unit tests assert: determinism; sensitivity to password/salt/parameters; rejection of invalid (m, t, p) ; avalanche smoke (single-bit password flips change roughly half of output bits in a small sample); sequential/parallel lane fill equivalence for $p = 2$; encode/verify round-trips; and parser negatives (B3 strings, whitespace, leading zeros). Avalanche tests are smoke-scale, not statistical cryptanalysis.

10.4 Vector methodology

Frozen Toy vectors pin digests for carefully chosen inputs (empty password / zero salt; short password with incrementing salt; two-lane case). Vectors are normative for implementers: a port is incorrect if it disagrees, regardless of campaign outcomes. Seed digests are published as informative aids for debugging partial implementations.

11 Results

11.1 Conformance vectors

Three frozen Toy vectors pin digests for empty password / zero salt; short password with incrementing salt; and a two-lane case. Full digests appear in Table 3.

Vector	Params	hashHex
1 empty / zero salt	m=1024,t=1,p=1	63def3608445d473b3d1ee9056747a6c481a668d085c96de2286bae4b57aa04a
2 password / 00..0f	m=1024,t=1,p=1	7e1916d2329e65ccb5c8e211c25c31efdd381cec765d721b35c2878e99ce9e08
3 x / 16x0x42	m=1024,t=1,p=2	3b050e341b18490362a602b8194c1702e7914d3a3f5b42225876cb75d56c0c13

Table 3: Frozen ForgeHash-X v0 toy digests (full 32-byte hex).

Automated tests assert seed, hash, and encoded string equality against these vectors. Encoded forms and seed digests are reproduced in Appendices A and B.

11.2 Functional property tests

Unit tests in `tests/ForgeHash.X.Tests` check the properties listed in Section 10. In the present work we report that these tests pass on the reference under Release configuration as exercised during the 2026-07-20 research snapshot; we do not enumerate per-test timing. Parallel equivalence for $p = 2$ is particularly important: it validates that lane barriers and synchronization constraints are implemented consistently with the sequential schedule.

11.3 Empirical uniqueness campaigns

On 2026-07-20, Collision Lab campaigns were run with $N = 100,000$ each.

11.3.1 Toy cost (1024 KiB, $t = 1$, $p = 1$)

Campaign	N	Collisions	Tracks
Random password+salt pairs	100 000	0	final hash
Nearby password bit-flips	100 000	0	final hash
Distinct salts (fixed password)	100 000	0	hash + seed
Distinct passwords (fixed salt)	100 000	0	hash + seed

Table 4: ForgeHash-X Toy uniqueness smoke hunts (2026-07-20).

11.3.2 Higher-memory smoke (4096 KiB, $t = 1$, $p = 1$)

Campaign	N	Collisions	Tracks
Distinct passwords (fixed salt)	100 000	0	hash + seed

Table 5: ForgeHash-X 4 MiB DistinctPasswords smoke hunt (2026-07-20).

Interpretation. Across 5×10^5 samples (four Toy campaigns plus one 4 MiB DistinctPasswords hunt), no accidental final-hash collision (and no seed collision where tracked) was observed. This is encouraging **consistency** evidence for the reference as memory grows beyond Toy. It is **not** a birthday-bound search at cryptographic digest width, **not** an adversarial structured-input study, and **not** informative about Interactive/Sensitive-scale costs (nor a full campaign matrix at 4 MiB).

11.4 Throughput (informal)

Mass runs at Toy reported on the order of approximately 2000 hashes/s on the author’s machine. This figure is **dominated by the 1 MiB Toy matrix**. It must not be read as “ForgeX is faster than

BLAKE3”: ForgeHash-B3 Development uses 8 MiB matrices (about $8 \times$ more memory). Matched- (m, t, p) benchmarks remain future work.

12 Discussion

12.1 Interpreting a clean uniqueness log

It is tempting—especially in a research narrative—to treat zero collisions in 5×10^5 samples as a milestone bordering on validation. We resist that temptation. Under a random-oracle heuristic for 256-bit digests, the expected number of collisions in 10^5 samples is negligible; thus the campaigns primarily demonstrate that the implementation is not pathologically broken on those distributions. The inclusion of seed tracking in DistinctSalts and DistinctPasswords campaigns adds a useful debugging signal: had seeds collided while final hashes differed (or vice versa), we would have localized bugs differently. Observing zeros on both tracks is consistent with a healthy reference, nothing more.

The 4 MiB DistinctPasswords hunt extends the smoke beyond Toy without claiming a comprehensive higher-cost matrix. We observe continuity of functional separation as m quadruples, which is the appropriate level of inference.

12.2 Comparison with ForgeHash-B3

Table 6 summarizes structural differences.

Aspect	ForgeHash-B3	ForgeHash-X v0
Encoded id / version	forgeh / v=1	forgehx / v=0
Primitive	BLAKE3 + ForgeMix	ForgeX sponge + ForgePerm
Block size	1024 bytes	512 bytes
Cross-lane cadence	every 32 blocks	every 16 blocks
External hash dep.	Yes (BLAKE3)	None
Stability	Frozen vectors + ports	Sandbox; may change
Intended use	Conformance / research	Custom-primitive research

Table 6: High-level comparison of B3 and X.

B3’s reliance on BLAKE3 [7] concentrates cryptanalytic trust in a widely studied primitive while still leaving the custom memory graph and ForgeMix as research surface. X inverts that allocation of novelty: the memory graph remains research-grade, but the primitive is also novel. Consequently, X is strictly earlier in any responsible adoption timeline than B3—and B3 itself is not a production recommendation in this project’s documentation.

Performance comparisons that ignore matrix size are actively misleading. Our informal 2000 H/s Toy observation and any B3 Development timings at 8 MiB answer different questions. We therefore decline comparative speed claims in this paper.

12.3 Position relative to OWASP guidance

OWASP’s Password Storage Cheat Sheet [6] recommends established memory-hard functions—especially Argon2id—with costs tuned to threat models, and it discourages home-grown password hashing for production. The present work is aligned with that guidance in an unusual way: by publishing a home-grown construction **as a non-production research artifact**, we aim to channel curiosity into reviewable specification work rather than into silent deployment. If readers take only one operational sentence from this paper, it should be: prefer Argon2id, scrypt, bcrypt, or platform password APIs for real password storage.

12.4 On scientific style and “padding”

Academic PHF papers sometimes oscillate between terse RFC-like specification and expansive security discussion. This document intentionally leans verbose in its prose framing—motivation, hedges, threats to validity—while keeping algorithms and vectors exact. Verbosity must not be mistaken for additional evidence. Where we lack GPU trials, we say so; where we lack proofs, we say so.

13 Limitations and Ethics

13.1 Technical limitations

1. **Young primitive.** ForgePerm has no third-party cryptanalysis.
2. **Sandbox versioning.** $v=0$ may break digests without migration.
3. **Cost confusion.** Toy is for tests; shipping Toy in an auth path would be reckless.
4. **Over-interpretation of uniqueness hunts.** Clean lab runs are easy to misread as “collision-resistant.”
5. **No GPU/ASIC study.** We report no attacker-cost measurements on non-CPU platforms.
6. **No TMTO theorem.** Memory-hardness is heuristic and structural, not proven.
7. **Side channels.** Data-dependent addressing is in scope for future evaluation but unevaluated here.
8. **Single reference language.** .NET 9 is authoritative for $v0$; other languages are future ports.

13.2 Ethical considerations

Password hashing research can be misapplied as marketing. We explicitly forbid (as authors of this document) any representation of ForgeHash-X as a “bcrypt killer,” “Argon2 replacement,” or similarly hyperbolic product claim. The ethical posture of the project is: publish mechanisms and vectors so that criticism is possible; keep algorithm identifiers and warnings loud; and direct practitioners to reviewed standards [1], [2], [6], [9].

Dual-use concerns are limited: the work does not introduce novel attack techniques against deployed PHFs; it introduces a new experimental verifier family that should not be deployed. The principal harm mode is premature adoption, which we attempt to mitigate through warnings, non-compatible encoding, and frank limitations.

Important. Do not store production passwords with ForgeHash-X or ForgeHash-B3 without extensive independent review. Prefer Argon2id, scrypt, bcrypt, or platform password APIs.

14 Future Work

1. Matched-cost benchmarks versus B3 and Argon2id under identical (m, t, p) and hardware conditions.
2. Dedicated ForgePerm known-answer tests and structural cryptanalysis (differential, linear, rotational, and algebraic probes).
3. Broader higher-cost uniqueness and avalanche campaigns (more kinds at 4–64 MiB; matched B3 Development).
4. Language ports against `implementers/x0`, with CI vector gates.
5. Optional stronger profiles if and when the primitive survives review—always under a new version tag, never by silently reinterpreting `v=0`.
6. Independent external analysis (essential before any production discussion).
7. Side-channel measurement plans for data-dependent fills, if the line continues.
8. Clarifying resource-limit policies for untrusted encoded parameters in online protocols.

15 Conclusion

ForgeHash-X v0 is a clean-sheet experimental password hashing construction: a custom sponge (ForgeX), an ARX permutation (ForgePerm), and a memory-hard fill with explicit domain separation and canonical encoding `$forgehx$v=0$m,t,p$salt$hash$`. A .NET 9 reference (ForgeHash.X.Core) with no BLAKE3 dependency, frozen Toy vectors ($m = 1024$, $t = 1$, $p = 1$), and 5×10^5 uniqueness samples (including a 4 MiB DistinctPasswords hunt) with zero observed collisions provide an early empirical baseline for specification stability and functional separation. Informal throughput on the order of 2000 H/s at Toy is apparatus context only and is not comparable to ForgeHash-B3 Development at 8 MiB.

The work is offered as an open research artifact—not as a replacement for Argon2id or other reviewed password KDFs. In the present work we have attempted to practice non-claim discipline: to specify precisely, measure modestly, and warn loudly. Heretofore the ForgeHash program has treated conformance tooling as part of the science; ForgeHash-X extends that culture to a fully custom primitive stack. Whether ForgePerm deserves to exist beyond a sandbox is a question for future cryptanalysis, not for this paper’s empirical smokes.

Important. ForgeHash-X is experimental research software. Digests are not compatible with ForgeHash-B3. Do not use either construction for production password storage without extensive independent review.

16 Acknowledgments

This paper documents work in the ForgeHash repository (<https://github.com/ThomasBeHappy/Forgehash>). Empirical campaigns used the shared Collision Lab / `CollisionCampaign` tooling developed for the ForgeHash-B3 research track. The author thanks, in advance, any reviewers and implementers who file precise vector mismatches or cryptanalytic observations; such feedback is the intended purpose of publishing a $v=0$ sandbox.

17 Extended Comparative Discussion

It is customary, in manuscripts that introduce a new password-based construction, to situate the proposal not only against its immediate sibling (here, ForgeHash-B3) but also against the broader ecology of password storage advice that practitioners actually encounter. OWASP’s password storage guidance [6] repeatedly emphasizes **reviewed** memory-hard functions—most prominently Argon2id—with carefully chosen memory and time parameters. The present work does not dispute that guidance. On the contrary: the ethical stance of ForgeHash-X v0 is that **experimental** constructions should be published with enough engineering completeness to be attacked, and with enough rhetorical restraint to avoid being mistaken for drop-in replacements.

Nevertheless, a comparative vocabulary is useful. Relative to PBKDF2 [8], ForgeHash-X (like scrypt, Argon2, and B3) attempts to raise the cost of large-scale guessing by consuming RAM, not merely by iterating a fast hash. Relative to bcrypt [9], which is CPU-hard and historically valuable but not designed as a multi-megabyte memory-hard function, ForgeHash-X follows the post-scrypt tradition. Relative to Argon2 [2], ForgeHash-X shares lane/slice synchronization **ideas** but substitutes a custom sponge for Blake2b and adopts different block geometry and addressing constants. Relative to Balloon [4] and Catena [5], which explore alternative graphs and proof-oriented framings, ForgeHash-X is more engineering-first: frozen vectors, parsers, and labs precede theorems.

One may therefore classify ForgeHash-X, at the present stage of maturity, as a **constructive probe**: an artifact intended to make the cost of designing a full-stack custom PHF visible, including the social cost of resisting hype when uniqueness hunts return zero collisions.

17.1 On the rhetoric of empirical uniqueness

The literature on birthday attacks and hash-function balance [17] reminds us that the absence of collisions in N samples, for digest width w bits, is unsurprising whenever N is far below $2^{w/2}$. For $w = 256$ (32-byte digests), 2^{128} is astronomically larger than 5×10^5 . Hence the campaigns reported herein are best understood as **engineering smoke tests**—they catch catastrophic implementation bugs, accidental truncation, or pathological constant folding—rather than as evidence of collision resistance in the cryptographic sense. We labor this point because zero is a psychologically powerful number, and research artifacts that print large tables of zeros risk being screenshot without their caveats.

17.2 On throughput narratives

Informal throughput observations (2000 hashes/s at Toy) are reported because operators of Collision Lab naturally notice them. They are **not** BenchmarkDotNet results, **not** matched against Argon2id or B3 Development, and **not** evidence that ForgePerm is “faster than BLAKE3.” Memory size dominates wall-clock cost in these constructions. A fair comparison would fix $(m, t, p, \text{outputLength})$, pin CPU affinity, disable turbo if required by the methodology, warm caches consistently, and report distributions rather than a single lab HUD number. That methodology is left as future work precisely so that the present paper does not accidentally invent a performance crown.

18 Algorithmic Cost Sketch

Although we advance no TMTO theorem, a back-of-the-envelope cost sketch helps set expectations for implementers and reviewers.

Let $B = 512$ denote the block size in bytes and let $\text{blockCount} = (m \times 1024)/B$. Under t passes and p lanes, the fill performs, to first order, $\Theta(t \times \text{blockCount})$ block mixes, each mix applying ForgePerm four times (once per 16-word chunk). Initialization performs $2p$ XOF expansions of 512 bytes. Finalization performs a constant number of block reads per lane plus two sponge calls (hash + XOF). Seed derivation is a single sponge hash of password and salt material.

Thus sequential work scales roughly linearly in m and t , and the resident memory is approximately m KiB (plus implementation overhead). Parallelism p does not reduce total work; it redistributes it across lanes with barriers, trading wall-clock time for aggregate RAM and synchronization. This is the same qualitative picture as Argon2-style designs [2], [15], and it is why Toy ($m = 1024$) feels brisk while 4096 KiB DistinctPasswords feels like a more serious smoke.

We emphasize again that asymptotic sketches are not substitutes for lower bounds against parallel adversaries with cheap memory.

19 Worked Example Walkthrough

To make the construction less abstract, consider Vector 2 at a narrative level (normative bytes remain those in Appendices A–B).

1. **Parameters.** $m = 1024$, $t = 1$, $p = 1$, `outputLength=32`, salt `00..0f`, password the UTF-8 string `password`.
2. **Seed material.** Family id 0, parameters, password length and bytes, salt length and bytes are concatenated little-endian and absorbed under `ForgeX/v0/seed`, yielding the 32-byte seed listed in Appendix B.
3. **Initialization.** With one lane, blocks 0 and 1 are filled by `ForgeX/v0/expand` XOFs tagged with lane and index.
4. **Fill.** A single pass of four slices writes the remaining blocks via `previous⊕reference` mixes. Because $p = 1$, references remain in-lane; `FastRange` still selects indices inside the allowed prefix.
5. **Finalization.** Selected blocks are XOR-folded; `ForgeX/v0/final` produces a root; `ForgeX/v0/output` expands to the 32-byte digest in Section Results.
6. **Encoding.** Salt and digest are Base64-encoded without padding and inserted into the canonical `$forgehx$v=0$...` string of Appendix A.

An independent implementer who mismatches any endianness, domain tag, padding byte, or `FastRange` width will diverge at the seed or shortly thereafter. This is intentional: early divergence is easier to debug than a final-byte mismatch of unclear provenance.

20 Reproducibility and Artifact Checklist

In the spirit of artifact-evaluation culture (without claiming a formal badge), we list what this paper’s claims require to reproduce:

Claim class	Reproduction path
Toy digests	<code>dotnet test tests/ForgeHash.X.Tests</code> against <code>implementers/x0</code>
Determinism / avalanche / parallel equivalence	Same test project
Parser hostility	Encoding / parser tests in <code>ForgeHash.X.Tests</code>
Uniqueness smokes	Collision Lab + <code>XCollisionHasher</code> ; see Appendix C
Normative algorithm	<code>docs/forgehx/SPECIFICATION_X.md</code>
This manuscript	<code>docs/forgehx/paper/</code> Typst sources + <code>typst compile</code>

Table 7: Artifact checklist for claims made in this paper.

We do **not** claim bit-identical throughput reproduction across machines. We **do** claim bit-identical digests for the frozen vectors on a conforming implementation.

21 Speculative Failure Modes (Non-Exhaustive)

A responsible experimental paper should enumerate how the construction might fail once cryptanalysts look. The following are **hypotheses**, not findings:

1. **ForgePerm structural attacks.** Reduced-round distinguishers, rotational cryptanalysis, or weak alignment between the product-enhanced ARX step and the word permutation could emerge. Eight rounds may be too few—or wasteful—once studied.
2. **Sponge misuse.** Incorrect multi-rate padding, tag truncation, or reuse of a single sponge state across modes would be implementation failures with security impact; the reference tries to prevent them by construction.
3. **Addressing biases.** FastRange is statistically strong for integer mapping [16], but the **combination** of address-word mixing and allowed-region truncation might create exploitable non-uniformity in pathological parameter regimes.
4. **Finalization thinness.** XOR-folding a handful of blocks is a deliberate simplification; an adversary who could control mid-state blocks more than we expect might aim at the fold. We have no proof that they cannot.
5. **Social failure.** The most likely failure mode in practice is not a novel cryptanalytic break but a developer deploying Toy parameters because they are the defaults in a research GUI.

Documenting these modes is not an admission that they are realized; it is an invitation to refute or confirm them.

22 Pedagogical Notes for Students

Course instructors who assign ForgeHash-X as a reading or porting exercise may wish to emphasize the following learning outcomes:

1. **Domain separation is not optional decoration.** Length-prefixed tags make transcript confusion attacks harder to stumble into.
2. **Canonical encoding is a security boundary.** Rejecting whitespace, leading zeros, and foreign algorithm ids is part of not being a confused deputy.
3. **Memory-hard design is a systems problem.** Barriers, lane counts, and block sizes interact with real CPUs and memory buses.
4. **Empirical zeros require verbal discipline.** Students should practice writing “no collisions in this sample set” instead of “collision-resistant.”
5. **Custom primitives are expensive socially.** Shipping a sponge is the easy part; earning trust is the hard part.

These notes are advisory and do not alter the normative specification.

23 On Versioning Philosophy

The choice of $v=0$ is deliberate semiotics. In some ecosystems, version zero signals “not ready.” In this project, it additionally signals **permission to break digests** when cryptanalysis or engineering review demands it. A future $v=1$ for X—if it ever exists—should be frozen with the same seriousness as ForgeHash-B3 v1: immovable vectors, migration notes, and a clear statement of what changed. Until then, any third-party storage of **forgeh**x digests is a category error.

24 Appendix A: Encoded Examples

This appendix reproduces canonical encoded strings for the frozen Toy vectors. These strings are normative for parser round-trips in the reference tests.

Vector	Encoded string
1	\$forgehx\$v=0\$m=1024,t=1,p=1\$AAAAAAAAAAAAAAAAAAAA\$Y97zYIRF1H0z0e6QVnR6bEgaZo0IXJbeIoa65LV6oEo
2	\$forgehx\$v=0\$m=1024,t=1,p=1\$AAECAwQFBgcICQoLDA00Dw\$fhkW0jKeZcy1y0IRwlwx7904H0x2XXIbNcKHjpn0ngg
3	\$forgehx\$v=0\$m=1024,t=1,p=2\$QkJCQkJCQkJCQkJCQkJCQg\$0wUONBsYSQNipgK4GUwXAueRTTo/W0IiWHbLddVsDBM

Table 8: Canonical encoded toy vectors.

It is worth noting that vector 3 uses $p = 2$ while retaining Toy memory and iteration settings, exercising multi-lane encoding and verification paths.

25 Appendix B: Seed Digests (Toy Vectors)

Seed digests are informative for implementers debugging partial stacks (e.g., seed derivation correct but fill incorrect). They are not sufficient verifiers by themselves.

Vector	seedHex
1	03ed1feaa66bc1cc42f95f8f7c96cd25328702d92fdf01329110c6c26bddcbbe
2	638b0883e681208a10a5dcf4ff59c49a303afb9804fb892e5c07258af932192d
3	ae90ad206d3ade0e01c4f4501e28a7558102d2ebb1e5b4d28ced4b3259dc0a14

Table 9: 32-byte seeds for toy vectors (informative for implementers).

26 Appendix C: Reproduce Campaigns

The following commands reproduce unit tests and launch Collision Lab from a repository checkout with a standard .NET 9 SDK:

```
dotnet test tests/ForgeHash.X.Tests -c Release
dotnet run --project src/ForgeHash.CollisionLab -c Release
```

In the lab: select **ForgeHash-X**, preset **Toy**, campaign as desired, $N = 100000$, workers approximately equal to CPU count. For the 4 MiB smoke, set memory to 4096 KiB while leaving $t = 1$, $p = 1$, and run DistinctPasswords with the same N .

Normative specification: docs/forgex/SPECIFICATION_X.md in the repository. Living empirical notes: docs/forgex/RESEARCH_NOTES.md. Campaign dates in this paper refer to 2026-07-20.

Hardware and OS details of the author’s informal 2000 H/s observation are intentionally not elevated to a benchmark claim; reproducers should record their own machine context if they publish timings.

27 Appendix D: Ethical and Responsible Use Statement

Password hashing research can be misapplied. This work is published to enable scrutiny of a custom construction and to practice full-stack specification discipline (vectors, parsers, labs). It must not be marketed as a secure password KDF. Any deployment discussion requires independent cryptanalysis, side-channel evaluation, and comparison against established standards under realistic threat models [6].

Responsible use includes:

1. Keeping `forgeh` / `v=0` labels intact in forks and papers.
2. Not presenting Toy-cost figures as deployment recommendations.
3. Not comparing unmatched memory profiles as evidence of primitive superiority.
4. Preferring Argon2id, scrypt, bcrypt, or platform password APIs in production systems.
5. Reporting vulnerabilities in the specification or reference via the project’s usual channels rather than silently deploying patches that change digests without version bumps.

The author affirms that the empirical collision counts reported herein are those observed in the stated Collision Lab campaigns on 2026-07-20, and that no GPU benchmark tables or formal security proofs have been omitted because they were unfavorable—rather, they were never produced for this snapshot.

28 Appendix E: Glossary and Notation Table

Symbol / term	Meaning
m	Memory parameter in KiB (<code>memoryKiB</code>).
t	Iteration / pass count.
p	Parallelism / lane count.
<code>outputLength</code>	Digest length in bytes (16...64 in sandbox).
R, C	Sponge rate and capacity (each 8 words / 512 bits in ForgeX).
S	16-word ForgeX / ForgePerm state.
ForgeX	Custom sponge primitive (absorb / pad / squeeze).
ForgePerm	8-round ARX permutation on $16 \times \text{u64}$.
ForgeHash-X	Full PHF construction; algorithm id <code>forgehx</code> , version <code>v=0</code> .
ForgeHash-B3	BLAKE3-based sibling line; algorithm id <code>forgeh</code> , version <code>v=1</code> .
Toy	Profile $m = 1024$, $t = 1$, $p = 1$, 32-byte output, 16-byte salt.
Block	512-byte memory unit ($64 \times \text{u64}$) in the X fill.
Lane / slice / pass	Argon2-like fill organization; 4 slices per pass.
FastRange	Lemire-style map: high 64 bits of $x \times n$ in 128-bit product.
Domain tag	Length-prefixed ASCII string absorbed before mode data.
Encoded string	<code>\$forgehx\$v=0\$m,t,p\$salt\$hash\$</code> canonical form.
Uniqueness hunt	Sample-set collision smoke; not a birthday-bound proof.
$a \times b$	Wrapping <code>u64</code> product unless a wider product is stated.
$a \oplus b$	Bitwise exclusive-or.
$x \parallel y$	Byte-string concatenation.
LE32, LE64	Little-endian integer encodings.

Table 10: Glossary and notation for ForgeHash-X v0.

Readers encountering dollar-prefixed algorithm strings in prose should recall that Typst math mode is avoided for those tokens; normative examples appear in `#raw` form throughout this document and in Appendix A.

References

- [1] C. Percival, “Stronger Key Derivation via Sequential Memory-Hard Functions,” in *BSDCan*, 2009. [Online]. Available: <https://www.tarsnap.com/scrypt/scrypt.pdf>
- [2] A. Biryukov, D. Dinu, and D. Khovratovich, “Argon2: New Generation of Memory-Hard Functions for Password Hashing and Other Applications,” in *IEEE European Symposium on Security and Privacy*, 2016, pp. 292–302. doi: 10.1109/EuroSP.2016.31.
- [3] Password Hashing Competition, “Password Hashing Competition.” [Online]. Available: <https://www.password-hashing.net/>
- [4] D. Boneh, H. Corrigan-Gibbs, and S. Schechter, “Balloon Hashing: A Memory-Hard Function Providing Provable Protection Against Sequential Attacks,” in *ASIACRYPT*, Springer, 2016, pp. 220–248. doi: 10.1007/978-3-662-53887-6_8.
- [5] C. Forler, S. Lucks, and J. Wenzel, “Catena: A Memory-Consuming Password-Scrambling Framework,” in *IACR Cryptology ePrint Archive*, 2013.
- [6] OWASP Foundation, “Password Storage Cheat Sheet.” [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
- [7] J. O'Connor, J.-P. Aumasson, S. Neves, and Z. Wilcox-O'Hearn, “BLAKE3 one function, fast everywhere.” [Online]. Available: <https://github.com/BLAKE3-team/BLAKE3-specs/blob/master/blake3.pdf>
- [8] B. Kaliski, “PKCS~#5: Password-Based Cryptography Specification Version 2.0,” technical report RFC 2898, 2000. doi: 10.17487/RFC2898.
- [9] N. Provos and D. Mazières, “A Future-Adaptable Password Scheme,” in *USENIX Annual Technical Conference*, 1999.
- [10] G. Bertoni, J. Daemen, M. Peeters, and G. V. Assche, “Cryptographic sponge functions,” *Submission to NIST (Round 3)*, 2011, [Online]. Available: <https://keccak.team/files/CSF-0.1.pdf>
- [11] G. Bertoni, J. Daemen, M. Peeters, and G. V. Assche, “The Keccak SHA-3 submission,” 2011, [Online]. Available: <https://keccak.team/files/Keccak-submission-3.pdf>
- [12] P. Rogaway, “Nonce-Based Symmetric Encryption,” in *FSE*, Springer, 2004, pp. 348–359.
- [13] D. J. Bernstein, “ChaCha, a variant of Salsa20,” in *Workshop Record of SASC*, 2008.
- [14] J.-P. Aumasson, L. Henzen, W. Meier, and R. C.-W. Phan, “SHA-3 proposal BLAKE,” 2010.
- [15] J. Alwen and J. Blocki, “Efficiently Computing Data-Independent Memory-Hard Functions,” in *CRYPTO*, Springer, 2016, pp. 241–271.
- [16] D. Lemire, “Fast Random Integer Generation in an Interval,” in *ACM Transactions on Modeling and Computer Simulation*, 2019.
- [17] M. Bellare and T. Kohno, “Hash Function Balance and Its Impact on Birthday Attacks,” in *EUROCRYPT*, Springer, 2004, pp. 401–418.